

Lecture 21

P, NP, and NP-completeness

Search vs Decision Problem

Search vs Decision Problem

Search problems are computational problems where we have to **find** a solution or inform

Search vs Decision Problem

Search problems are computational problems where we have to **find** a solution or inform that no solutions exist.

Search vs Decision Problem

Search problems are computational problems where we have to **find** a solution or inform that no solutions exist. For instance,

Search vs Decision Problem

Search problems are computational problems where we have to **find** a solution or inform that no solutions exist. For instance,

SEARCH_PATH: Given a graph G and vertices $u, v \in G$, find a path from u to v or inform if no such paths exist.

Search vs Decision Problem

Search problems are computational problems where we have to **find** a solution or inform that no solutions exist. For instance,

SEARCH_PATH: Given a graph G and vertices $u, v \in G$, find a path from u to v or inform if no such paths exist.

Decision problems are computational problems where we have to **decide** whether a solution exists.

Search vs Decision Problem

Search problems are computational problems where we have to **find** a solution or inform that no solutions exist. For instance,

SEARCH_PATH: Given a graph G and vertices $u, v \in G$, find a path from u to v or inform if no such paths exist.

Decision problems are computational problems where we have to **decide** whether a solution exists. For instance,

Search vs Decision Problem

Search problems are computational problems where we have to **find** a solution or inform that no solutions exist. For instance,

SEARCH_PATH: Given a graph G and vertices $u, v \in G$, find a path from u to v or inform if no such paths exist.

Decision problems are computational problems where we have to **decide** whether a solution exists. For instance,

DEC_PATH: Given a graph G and vertices $u, v \in G$, decide if a path from u to v exist.

Search vs Decision Problem

Search vs Decision Problem

Solving *DEC_PATH* via *SEARCH_PATH*:

Search vs Decision Problem

Solving *DEC_PATH* via *SEARCH_PATH*:

1. Solve *SEARCH_PATH* on (G, u, v) .

Search vs Decision Problem

Solving *DEC_PATH* via *SEARCH_PATH*:

1. Solve *SEARCH_PATH* on (G, u, v) .
2. Answer **Yes** or **No** depending on the answer from the 1st step.

Search vs Decision Problem

Solving *DEC_PATH* via *SEARCH_PATH*:

1. Solve *SEARCH_PATH* on (G, u, v) .
2. Answer **Yes** or **No** depending on the answer from the 1st step.

Observation:

Search vs Decision Problem

Solving *DEC_PATH* via *SEARCH_PATH*:

1. Solve *SEARCH_PATH* on (G, u, v) .
2. Answer **Yes** or **No** depending on the answer from the 1st step.

Observation: If *SEARCH_PATH* is polynomial-time solvable, then so is *DEC_PATH*.

Search vs Decision Problem

Solving *DEC_PATH* via *SEARCH_PATH*:

1. Solve *SEARCH_PATH* on (G, u, v) .
2. Answer **Yes** or **No** depending on the answer from the 1st step.

Observation:

Search vs Decision Problem

Solving *DEC_PATH* via *SEARCH_PATH*:

1. Solve *SEARCH_PATH* on (G, u, v) .
2. Answer **Yes** or **No** depending on the answer from the 1st step.

Observation: If *DEC_PATH* is not polynomial-time solvable, then so is *SEARCH_PATH*.

Search vs Decision Problem

Solving *DEC_PATH* via *SEARCH_PATH*:

1. Solve *SEARCH_PATH* on (G, u, v) .
2. Answer **Yes** or **No** depending on the answer from the 1st step.

Observation: If *DEC_PATH* is not polynomial-time solvable, then so is *SEARCH_PATH*.

We will focus on decision problems because:

Search vs Decision Problem

Solving *DEC_PATH* via *SEARCH_PATH*:

1. Solve *SEARCH_PATH* on (G, u, v) .
2. Answer **Yes** or **No** depending on the answer from the 1st step.

Observation: If *DEC_PATH* is not polynomial-time solvable, then so is *SEARCH_PATH*.

We will focus on decision problems because:

- They are mathematically simple.

Search vs Decision Problem

Solving *DEC_PATH* via *SEARCH_PATH*:

1. Solve *SEARCH_PATH* on (G, u, v) .
2. Answer **Yes** or **No** depending on the answer from the 1st step.

Observation: If *DEC_PATH* is not polynomial-time solvable, then so is *SEARCH_PATH*.

We will focus on decision problems because:

- They are mathematically simple.
- Lower bounds for decision problems implies lower bounds for search problems.

Search vs Decision Problem

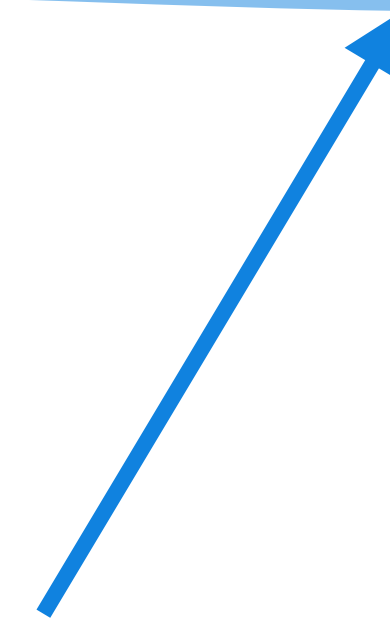
Solving *DEC_PATH* via *SEARCH_PATH*:

1. Solve *SEARCH_PATH* on (G, u, v) .
2. Answer **Yes** or **No** depending on the answer from the 1st step.

Observation: If *DEC_PATH* is not polynomial-time solvable, then so is *SEARCH_PATH*.

We will focus on decision problems because:

- They are mathematically simple.
- Lower bounds for decision problems implies lower bounds for search problems.



Complexity Class P

Complexity Class P

Defn: P is the set of decision problems that can be solved in **polynomial time**, i.e., $O(n^c)$, where $c \geq 0$ and n is the length of the input.

Complexity Class P

Defn: P is the set of decision problems that can be solved in **polynomial time**, i.e., $O(n^c)$, where $c \geq 0$ and n is the length of the input.

Some problems in P:

Complexity Class P

Defn: P is the set of decision problems that can be solved in **polynomial time**, i.e., $O(n^c)$, where $c \geq 0$ and n is the length of the input.

Some problems in P :

- $MULT = \{ \langle x, y, i \rangle \mid \text{The } i\text{th bit of } xy \text{ is } 1 \}$

Complexity Class P

Defn: P is the set of decision problems that can be solved in **polynomial time**, i.e., $O(n^c)$, where $c \geq 0$ and n is the length of the input.

Some problems in P:  Binary encoding of x , y , and i .

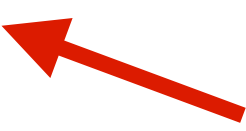
- $MULT = \{ \langle x, y, i \rangle \mid \text{The } i\text{th bit of } xy \text{ is } 1 \}$

Complexity Class P

Defn: P is the set of decision problems that can be solved in **polynomial time**, i.e., $O(n^c)$, where $c \geq 0$ and n is the length of the input.

Some problems in P:  Binary encoding of x, y , and i .

- $MULT = \{ \langle x, y, i \rangle \mid \text{The } i\text{th bit of } xy \text{ is } 1 \}$

 Decision problems can be expressed as sets.

Complexity Class P

Defn: P is the set of decision problems that can be solved in **polynomial time**, i.e., $O(n^c)$, where $c \geq 0$ and n is the length of the input.

Some problems in P :

Binary encoding of x, y , and i .

• $MULT = \{ \langle x, y, i \rangle \mid \text{The } i\text{th bit of } xy \text{ is } 1 \}$

• $PRIMES = \{ \langle x \rangle \mid x \text{ is a prime number} \}$

Decision problems can be expressed as sets.

Complexity Class P

Defn: P is the set of decision problems that can be solved in **polynomial time**, i.e., $O(n^c)$, where $c \geq 0$ and n is the length of the input.

Some problems in P:

Binary encoding of x, y , and i .

- $MULT = \{ \langle x, y, i \rangle \mid \text{The } i\text{th bit of } xy \text{ is } 1 \}$
- $PRIMES = \{ \langle x \rangle \mid x \text{ is a prime number} \}$
- $SORT = \{ \langle x_1, x_2, \dots, x_n, i \rangle \mid \text{The } i\text{th bit of sorted sequence } (x_{k_1}, x_{k_2}, \dots, x_{k_n}) \text{ is } 1 \}$

Decision problems can be expressed as sets.

Complexity Class P

Defn: P is the set of decision problems that can be solved in **polynomial time**, i.e., $O(n^c)$, where $c \geq 0$ and n is the length of the input.

Some problems in P :

Binary encoding of x, y , and i .

- $MULT = \{ \langle x, y, i \rangle \mid \text{The } i\text{th bit of } xy \text{ is } 1 \}$
- $PRIMES = \{ \langle x \rangle \mid x \text{ is a prime number} \}$
- $SORT = \{ \langle x_1, x_2, \dots, x_n, i \rangle \mid \text{The } i\text{th bit of sorted sequence } (x_{k_1}, x_{k_2}, \dots, x_{k_n}) \text{ is } 1 \}$

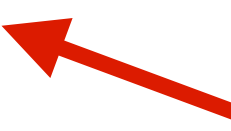
Decision problems can be expressed as sets.

Some problems not known to be in P :

Complexity Class P

Defn: P is the set of decision problems that can be solved in **polynomial time**, i.e., $O(n^c)$, where $c \geq 0$ and n is the length of the input.

Some problems in P:  Binary encoding of x, y , and i .

- $MULT = \{ \langle x, y, i \rangle \mid \text{The } i\text{th bit of } xy \text{ is } 1 \}$
 - $PRIMES = \{ \langle x \rangle \mid x \text{ is a prime number} \}$
 - $SORT = \{ \langle x_1, x_2, \dots, x_n, i \rangle \mid \text{The } i\text{th bit of sorted sequence } (x_{k_1}, x_{k_2}, \dots, x_{k_n}) \text{ is } 1 \}$
-  Decision problems can be expressed as sets.

Some problems not known to be in P:

- $GI = \{ \langle G_1, G_2 \rangle \mid \text{Graphs } G_1 \text{ and } G_2 \text{ are isomorphic} \}$

Complexity Class P

Defn: **P** is the set of decision problems that can be solved in **polynomial time**, i.e., $O(n^c)$, where $c \geq 0$ and n is the length of the input.

Some problems in **P**:

Binary encoding of x, y , and i .

- $MULT = \{ \langle x, y, i \rangle \mid \text{The } i\text{th bit of } xy \text{ is } 1 \}$
- $PRIMES = \{ \langle x \rangle \mid x \text{ is a prime number} \}$
- $SORT = \{ \langle x_1, x_2, \dots, x_n, i \rangle \mid \text{The } i\text{th bit of sorted sequence } (x_{k_1}, x_{k_2}, \dots, x_{k_n}) \text{ is } 1 \}$

Decision problems can be expressed as sets.

Some problems not known to be in **P**:

- $GI = \{ \langle G_1, G_2 \rangle \mid \text{Graphs } G_1 \text{ and } G_2 \text{ are isomorphic} \}$
- $FACTOR = \{ \langle x, l, u \rangle \mid x \text{ has a factor } y \text{ such that } l \leq y \leq u \}$

Complexity Class P

Complexity Class P

Why **P** captures efficiency:

Complexity Class P

Why **P** captures efficiency:

- Time bounds less than n do not allow to even read the input on Turing machines.

Complexity Class P

Why **P** captures efficiency:

- Time bounds less than n do not allow to even read the input on Turing machines.
- In practice, we find algorithms with time n^2, n^3 , etc. (not n^{30}, n^{100} , etc.).

Complexity Class P

Why **P** captures efficiency:

- Time bounds less than n do not allow to even read the input on Turing machines.
- In practice, we find algorithms with time n^2, n^3 , etc. (not n^{30}, n^{100} , etc.).
- Problems with first polytime algorithm with high complexity, say $O(n^{20})$, usually get

Complexity Class P

Why **P** captures efficiency:

- Time bounds less than n do not allow to even read the input on Turing machines.
- In practice, we find algorithms with time n^2, n^3 , etc. (not n^{30}, n^{100} , etc.).
- Problems with first polytime algorithm with high complexity, say $O(n^{20})$, usually get improved to, say $O(n^5)$.

Complexity Class NP

Complexity Class NP

Some problems not known to be in **P**:

Complexity Class NP

Some problems not known to be in **P**:

- $INDSET = \{ \langle G, k \rangle \mid G \text{ has an independent set of size } k \}$

Complexity Class NP

Some problems not known to be in **P**:

- $INDSET = \{ \langle G, k \rangle \mid G \text{ has an independent set of size } k \}$
- $HAMILTONIANCYCLE = \{ \langle G \rangle \mid G \text{ has a hamiltonian cycle} \}$

Complexity Class NP

Some problems not known to be in **P**:

- $INDSET = \{ \langle G, k \rangle \mid G \text{ has an independent set of size } k \}$
- $HAMILTONIANCYCLE = \{ \langle G \rangle \mid G \text{ has a hamiltonian cycle} \}$
- $GI = \{ \langle G_1, G_2 \rangle \mid \text{Graphs } G_1 \text{ and } G_2 \text{ are isomorphic} \}$

Complexity Class NP

Some problems not known to be in **P**:

- $INDSET = \{ \langle G, k \rangle \mid G \text{ has an independent set of size } k \}$
 - $HAMILTONIANCYCLE = \{ \langle G \rangle \mid G \text{ has a hamiltonian cycle} \}$
 - $GI = \{ \langle G_1, G_2 \rangle \mid \text{Graphs } G_1 \text{ and } G_2 \text{ are isomorphic} \}$
- 
- not polytime solvable,
but solutions are
verifiable in polytime.

Complexity Class NP

Some problems not known to be in **P**:

- $INDSET = \{ \langle G, k \rangle \mid G \text{ has an independent set of size } k \}$
 - $HAMILTONIANCYCLE = \{ \langle G \rangle \mid G \text{ has a hamiltonian cycle} \}$
 - $GI = \{ \langle G_1, G_2 \rangle \mid \text{Graphs } G_1 \text{ and } G_2 \text{ are isomorphic} \}$
- 
- not polytime solvable,
but solutions are
verifiable in polytime.

Informal Definition:

Complexity Class NP

Some problems not known to be in **P**:

- $INDSET = \{ \langle G, k \rangle \mid G \text{ has an independent set of size } k \}$
 - $HAMILTONIANCYCLE = \{ \langle G \rangle \mid G \text{ has a hamiltonian cycle} \}$
 - $GI = \{ \langle G_1, G_2 \rangle \mid \text{Graphs } G_1 \text{ and } G_2 \text{ are isomorphic} \}$
- 
- not polytime solvable,
but solutions are
verifiable in polytime.

Informal Definition: NP captures the set of decisions problems whose solutions can be

Complexity Class NP

Some problems not known to be in **P**:

- $INDSET = \{ \langle G, k \rangle \mid G \text{ has an independent set of size } k \}$
 - $HAMILTONIANCYCLE = \{ \langle G \rangle \mid G \text{ has a hamiltonian cycle} \}$
 - $GI = \{ \langle G_1, G_2 \rangle \mid \text{Graphs } G_1 \text{ and } G_2 \text{ are isomorphic} \}$
- 
- not polytime solvable,
but solutions are
verifiable in polytime.

Informal Definition: NP captures the set of decisions problems whose solutions can be verified efficiently.

Complexity Class NP

Complexity Class NP

Definition: A decision problem L is in **NP** if $\forall x \in \{0,1\}^*$,

Complexity Class NP

Definition: A decision problem L is in **NP** if $\forall x \in \{0,1\}^*$,

$x \in L \implies \exists$ a solution u that can be verified efficiently

Complexity Class NP

Definition: A decision problem L is in **NP** if $\forall x \in \{0,1\}^*$,

$x \in L \implies \exists$ a solution u that can be verified efficiently

$x \notin L \implies \exists$ no solution u that can be verified efficiently

Complexity Class NP

Definition: A decision problem L is in **NP** if $\forall x \in \{0,1\}^*$,

$$x \in L \implies \exists u \in \{0,1\}^{p(|x|)}, \text{ such that } A(x, u) = 1$$

$$x \notin L \implies \exists \text{ no solution } u \text{ that can be verified efficiently}$$

Complexity Class NP

Definition: A decision problem L is in **NP** if $\forall x \in \{0,1\}^*$,

$$x \in L \implies \exists u \in \{0,1\}^{p(|x|)}, \text{ such that } A(x, u) = 1$$

$$x \notin L \implies \forall u \in \{0,1\}^{p(|x|)}, A(x, u) = 0$$

Complexity Class NP

Definition: A decision problem L is in **NP** if there exists a polytime algorithm, say A , and a

$$x \in L \implies \exists u \in \{0,1\}^{p(|x|)}, \text{ such that } A(x, u) = 1$$

$$x \notin L \implies \forall u \in \{0,1\}^{p(|x|)}, A(x, u) = 0$$

Complexity Class NP

Definition: A decision problem L is in **NP** if there exists a polytime algorithm, say A , and a polynomial $p : \mathbb{N} \rightarrow \mathbb{N}$, such that $\forall x \in \{0,1\}^*$,

$$x \in L \implies \exists u \in \{0,1\}^{p(|x|)}, \text{ such that } A(x, u) = 1$$

$$x \notin L \implies \forall u \in \{0,1\}^{p(|x|)}, A(x, u) = 0$$

Complexity Class NP

Definition: A decision problem L is in **NP** if there exists a polytime algorithm, say A , and a polynomial $p : \mathbb{N} \rightarrow \mathbb{N}$, such that $\forall x \in \{0,1\}^*$,

$$x \in L \iff \exists u \in \{0,1\}^{p(|x|)}, \text{ such that } A(x, u) = 1$$

Complexity Class NP

Definition: A decision problem L is in **NP** if there exists a polytime algorithm, say A , and a polynomial $p : \mathbb{N} \rightarrow \mathbb{N}$, such that $\forall x \in \{0,1\}^*$,

$$x \in L \iff \exists u \in \{0,1\}^{p(|x|)}, \text{ such that } A(x, u) = 1$$

Note: If $x \in L$ and $u \in \{0,1\}^{p(|x|)}$ satisfy $A(x, u) = 1$

Complexity Class NP

Definition: A decision problem L is in **NP** if there exists a polytime algorithm, say A , and a polynomial $p : \mathbb{N} \rightarrow \mathbb{N}$, such that $\forall x \in \{0,1\}^*$,

$$x \in L \iff \exists u \in \{0,1\}^{p(|x|)}, \text{ such that } A(x, u) = 1$$

Note: If $x \in L$ and $u \in \{0,1\}^{p(|x|)}$ satisfy $A(x, u) = 1$, then u is called a **certificate** for x .

Complexity Class NP

Definition: A decision problem L is in **NP** if there exists a polytime algorithm, say A , and a polynomial $p : \mathbb{N} \rightarrow \mathbb{N}$, such that $\forall x \in \{0,1\}^*$,

$$x \in L \iff \exists u \in \{0,1\}^{p(|x|)}, \text{ such that } A(x, u) = 1$$

Note: If $x \in L$ and $u \in \{0,1\}^{p(|x|)}$ satisfy $A(x, u) = 1$, then u is called a **certificate** for x .

Two features of **NP**:

Complexity Class NP

Definition: A decision problem L is in **NP** if there exists a polytime algorithm, say A , and a polynomial $p : \mathbb{N} \rightarrow \mathbb{N}$, such that $\forall x \in \{0,1\}^*$,

$$x \in L \iff \exists u \in \{0,1\}^{p(|x|)}, \text{ such that } A(x, u) = 1$$

Note: If $x \in L$ and $u \in \{0,1\}^{p(|x|)}$ satisfy $A(x, u) = 1$, then u is called a **certificate** for x .

Two features of **NP**:

- Proposed solutions are encodable by a polynomially long binary string.

Complexity Class NP

Definition: A decision problem L is in **NP** if there exists a polytime algorithm, say A , and a polynomial $p : \mathbb{N} \rightarrow \mathbb{N}$, such that $\forall x \in \{0,1\}^*$,

$$x \in L \iff \exists u \in \{0,1\}^{p(|x|)}, \text{ such that } A(x, u) = 1$$

Note: If $x \in L$ and $u \in \{0,1\}^{p(|x|)}$ satisfy $A(x, u) = 1$, then u is called a **certificate** for x .

Two features of **NP**:

- Proposed solutions are encodable by a polynomially long binary string.
- There exists a polytime algorithm to check if a proposed solution is a correct solution.

Complexity Class NP

Complexity Class NP

Example:

Complexity Class NP

Example: $COMPOSITE = \{ \langle x \rangle \mid x \text{ is a composite number} \} \in \text{NP}.$

Complexity Class NP

Example: $COMPOSITE = \{ \langle x \rangle \mid x \text{ is a composite number} \} \in \text{NP}$.

Let A be a polytime algorithm and $p : \mathbb{N} \rightarrow \mathbb{N}$ be a polynomial.

Complexity Class NP

Example: $COMPOSITE = \{ \langle x \rangle \mid x \text{ is a composite number} \} \in \text{NP}$.

Let A be a polytime algorithm and $p : \mathbb{N} \rightarrow \mathbb{N}$ be a polynomial.

Then, A on input $\langle x, u \rangle$, where $u \in \{0,1\}^{p(|x|)}$:

Complexity Class NP

Example: $COMPOSITE = \{ \langle x \rangle \mid x \text{ is a composite number} \} \in \text{NP}$.

Let A be a polytime algorithm and $p : \mathbb{N} \rightarrow \mathbb{N}$ be a polynomial.

Then, A on input $\langle x, u \rangle$, where $u \in \{0,1\}^{p(|x|)}$:

- Treats u as an integer from 2 to $(x - 1)$.

Complexity Class NP

Example: $COMPOSITE = \{ \langle x \rangle \mid x \text{ is a composite number} \} \in \text{NP}$.

Let A be a polytime algorithm and $p : \mathbb{N} \rightarrow \mathbb{N}$ be a polynomial.

Then, A on input $\langle x, u \rangle$, where $u \in \{0,1\}^{p(|x|)}$:

- Treats u as an integer from 2 to $(x - 1)$.
- Outputs 1 if x is divisible by u , else outputs 0.

Complexity Class NP

Example: $COMPOSITE = \{ \langle x \rangle \mid x \text{ is a composite number} \} \in \text{NP}$.

Let A be a polytime algorithm and $p : \mathbb{N} \rightarrow \mathbb{N}$ be a polynomial.

Then, A on input $\langle x, u \rangle$, where $u \in \{0,1\}^{p(|x|)}$:

- Treats u as an integer from 2 to $(x - 1)$.
- Outputs 1 if x is divisible by u , else outputs 0.

Clearly,

Complexity Class NP

Example: $COMPOSITE = \{ \langle x \rangle \mid x \text{ is a composite number} \} \in \text{NP}$.

Let A be a polytime algorithm and $p : \mathbb{N} \rightarrow \mathbb{N}$ be a polynomial.

Then, A on input $\langle x, u \rangle$, where $u \in \{0,1\}^{p(|x|)}$:

- Treats u as an integer from 2 to $(x - 1)$.
- Outputs 1 if x is divisible by u , else outputs 0.

Clearly,

$$\langle x \rangle \in COMPOSITE \iff \exists u \in \{0,1\}^{p(|x|)}, \text{ such that } A(x, u) = 1.$$

Complexity Class NP

Example: $COMPOSITE = \{ \langle x \rangle \mid x \text{ is a composite number} \} \in \text{NP}$.

Let A be a polytime algorithm and $p : \mathbb{N} \rightarrow \mathbb{N}$ be a polynomial.

Then, A on input $\langle x, u \rangle$, where $u \in \{0,1\}^{p(|x|)}$:

- Treats u as an integer from 2 to $(x - 1)$.
- Outputs 1 if x is divisible by u , else outputs 0.

Clearly,

$$\langle x \rangle \in COMPOSITE \iff \exists u \in \{0,1\}^{p(|x|)}, \text{ such that } A(x, u) = 1.$$



P vs NP

P **vs** NP

Claim: $P \subseteq NP$.

P **vs** NP

Claim: $P \subseteq NP$.

Proof Idea: Problems that can be solved easily can also be verified easily.

P **vs** NP

Claim: $P \subseteq NP$.

Proof Idea: Problems that can be solved easily can also be verified easily.

Is $NP \subseteq P$?

P **vs** NP

Claim: $P \subseteq NP$.

Proof Idea: Problems that can be solved easily can also be verified easily.

Is $NP \subseteq P$?

A million dollar question!

NP-Completeness

NP-Completeness

Diversity in NP:

NP-Completeness

Diversity in NP:

- *PRIME* is solvable in polynomial-time.

NP-Completeness

Diversity in NP:

- *PRIME* is solvable in polynomial-time.
- *GI* is solvable in quasi-polynomial time ($O(n^{\log^c n})$).

NP-Completeness

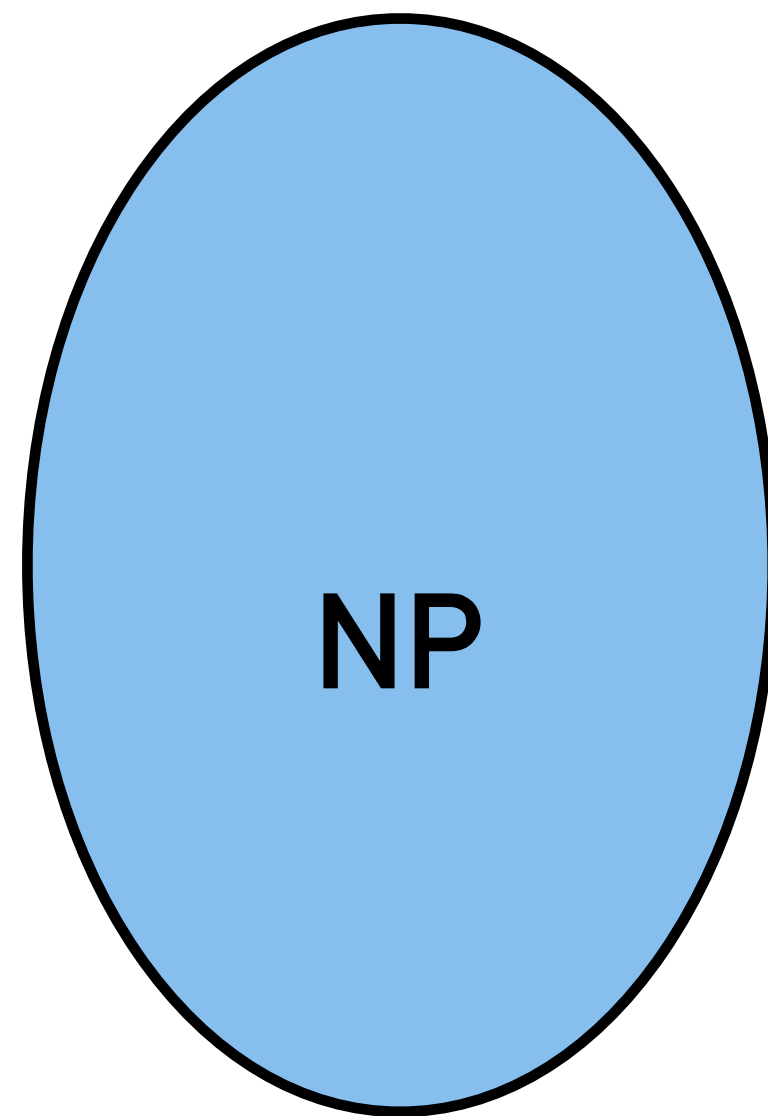
Diversity in NP:

- *PRIME* is solvable in polynomial-time.
- *GI* is solvable in quasi-polynomial time ($O(n^{\log^c n})$).
- *INDSET* is solvable in $O(1.1996^n)$.

NP-Completeness

Diversity in NP:

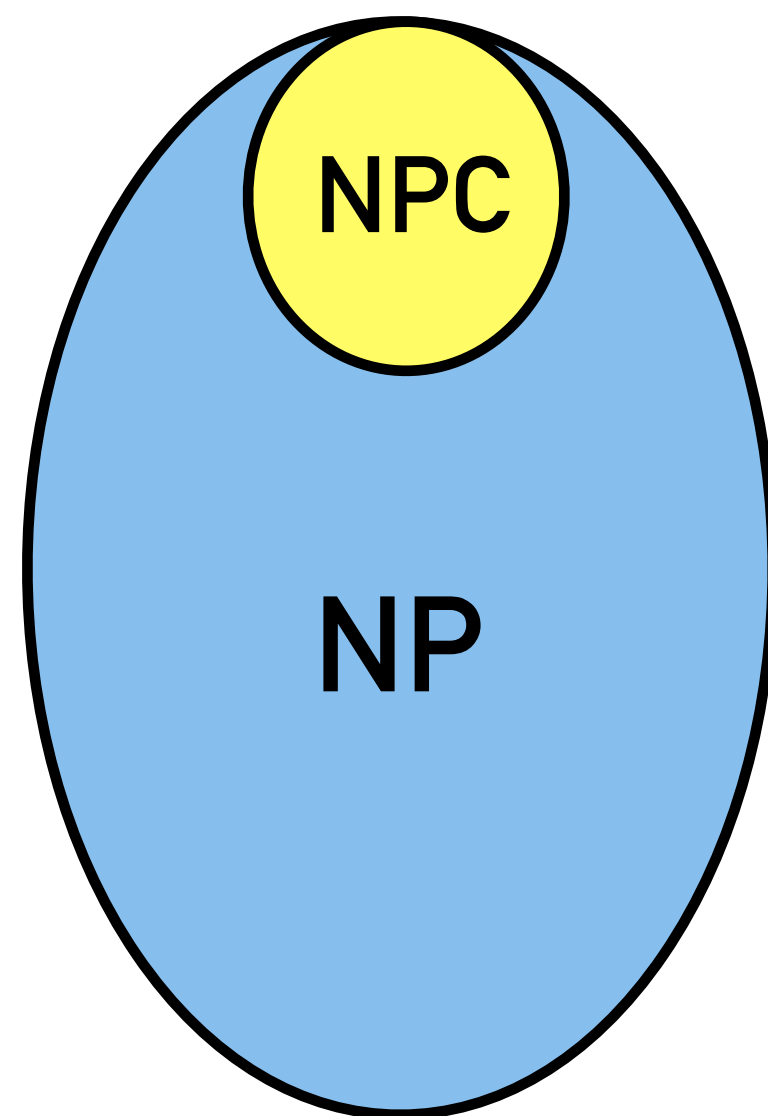
- *PRIME* is solvable in polynomial-time.
- *GI* is solvable in quasi-polynomial time ($O(n^{\log^c n})$).
- *INDSET* is solvable in $O(1.1996^n)$.



NP-Completeness

Diversity in NP:

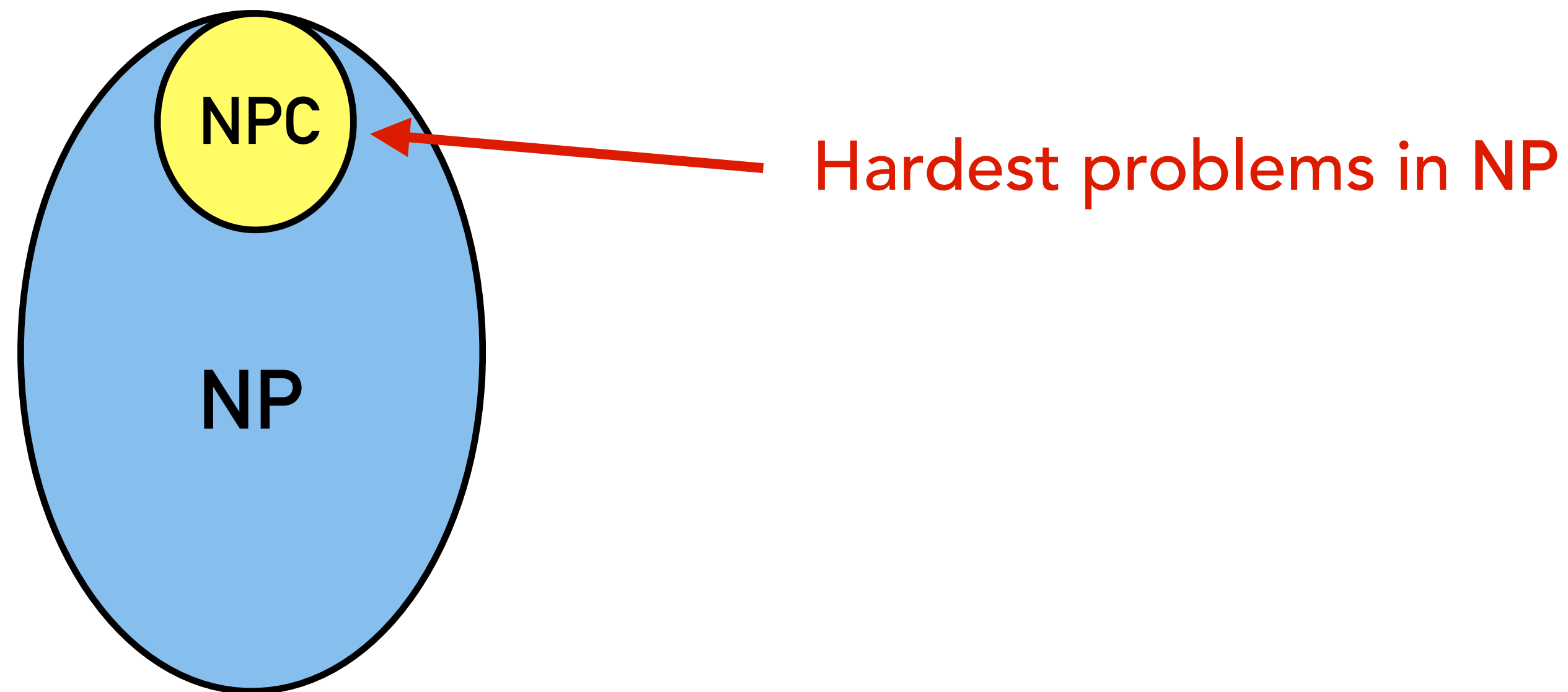
- *PRIME* is solvable in polynomial-time.
- *GI* is solvable in quasi-polynomial time ($O(n^{\log^c n})$).
- *INDSET* is solvable in $O(1.1996^n)$.



NP-Completeness

Diversity in NP:

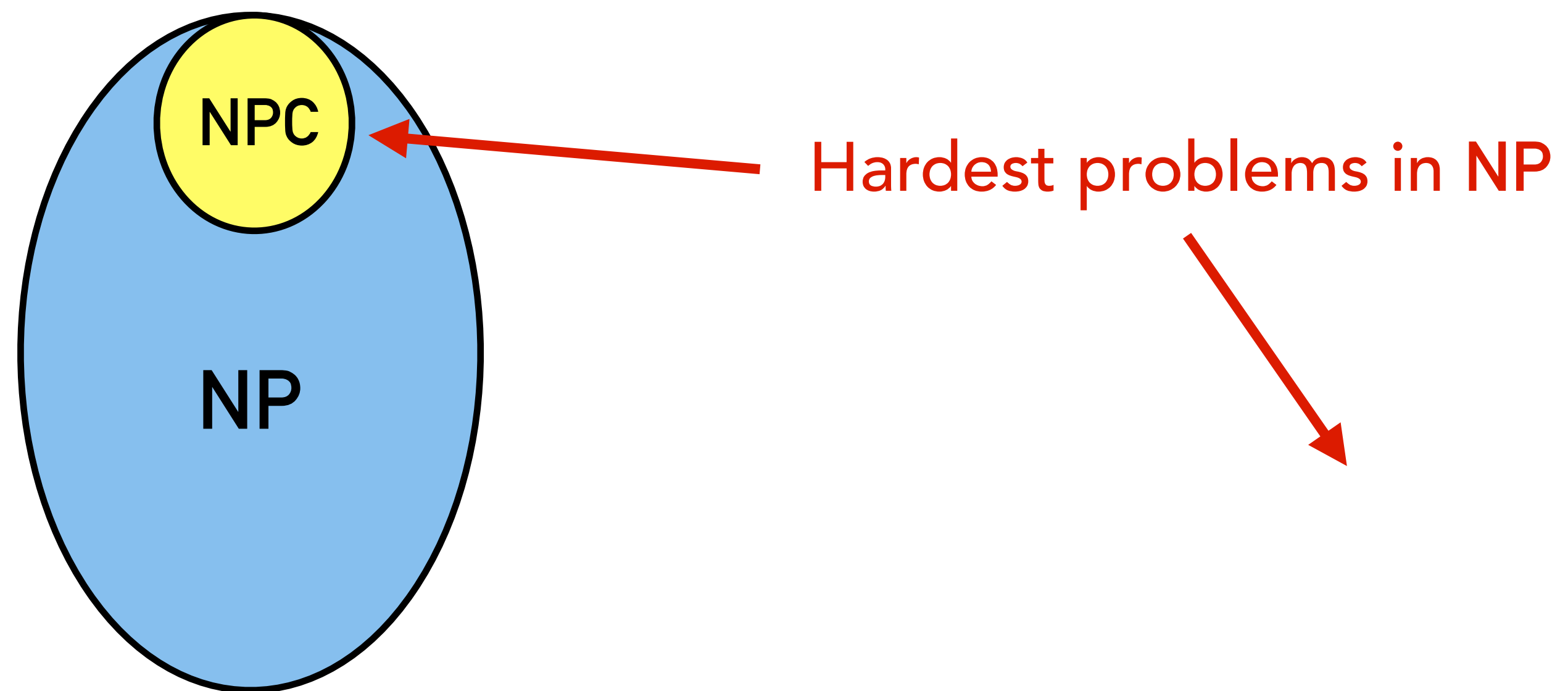
- *PRIME* is solvable in polynomial-time.
- *GI* is solvable in quasi-polynomial time ($O(n^{\log^c n})$).
- *INDSET* is solvable in $O(1.1996^n)$.



NP-Completeness

Diversity in NP:

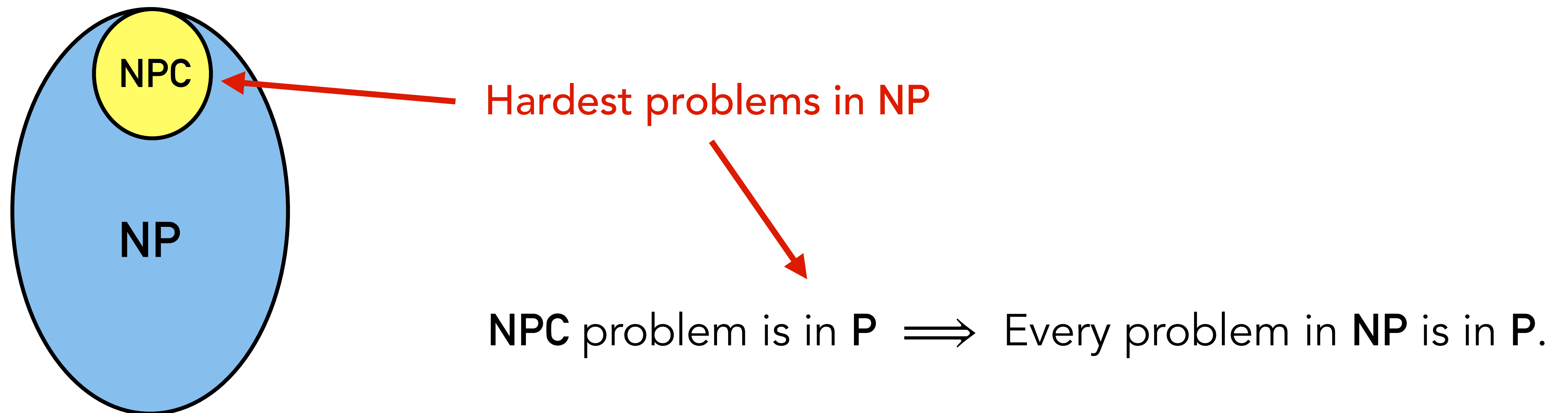
- *PRIME* is solvable in polynomial-time.
- *GI* is solvable in quasi-polynomial time ($O(n^{\log^c n})$).
- *INDSET* is solvable in $O(1.1996^n)$.



NP-Completeness

Diversity in NP:

- *PRIME* is solvable in polynomial-time.
- *GI* is solvable in quasi-polynomial time ($O(n^{\log^c n})$).
- *INDSET* is solvable in $O(1.1996^n)$.



NP-Completeness and NP-Hardness

NP-Completeness and NP-Hardness

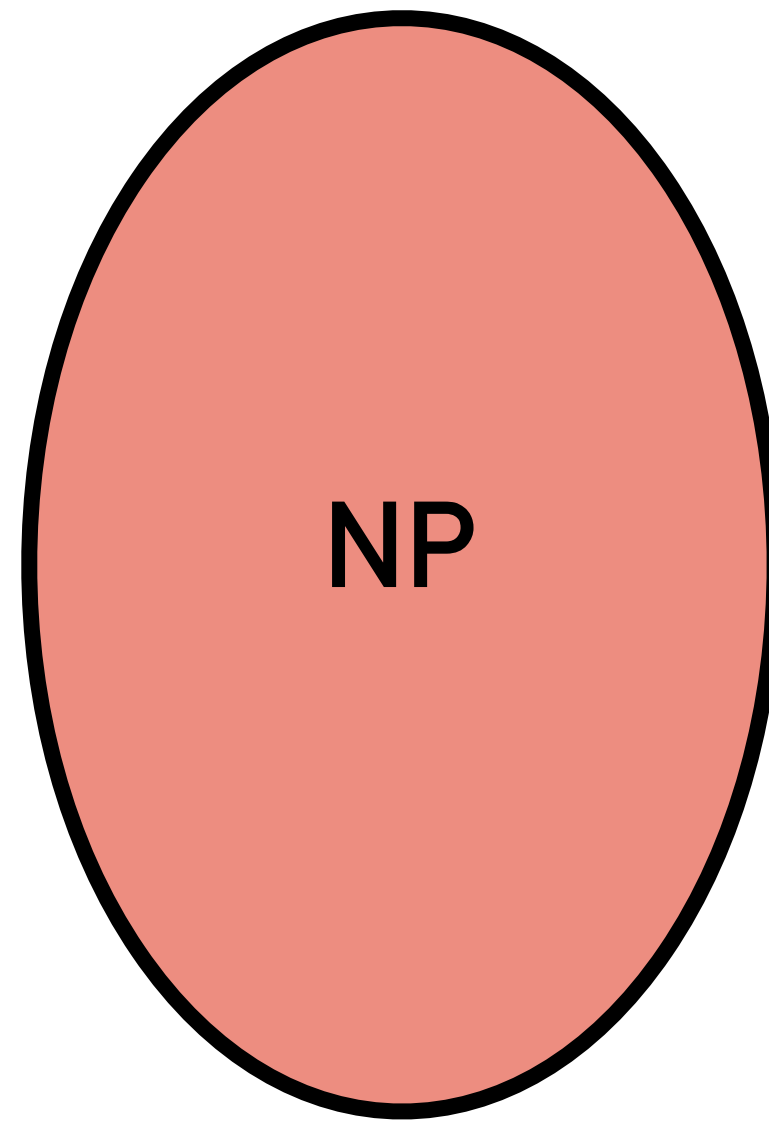
Defn: A decision problem L in **NP** is called **NP-complete** if L' can be “reduced” to L , denoted by

NP-Completeness and NP-Hardness

Defn: A decision problem L in **NP** is called **NP-complete** if L' can be “reduced” to L , denoted by $L' \leq_p L$ in polynomial-time, for every $L' \in \text{NP}$.

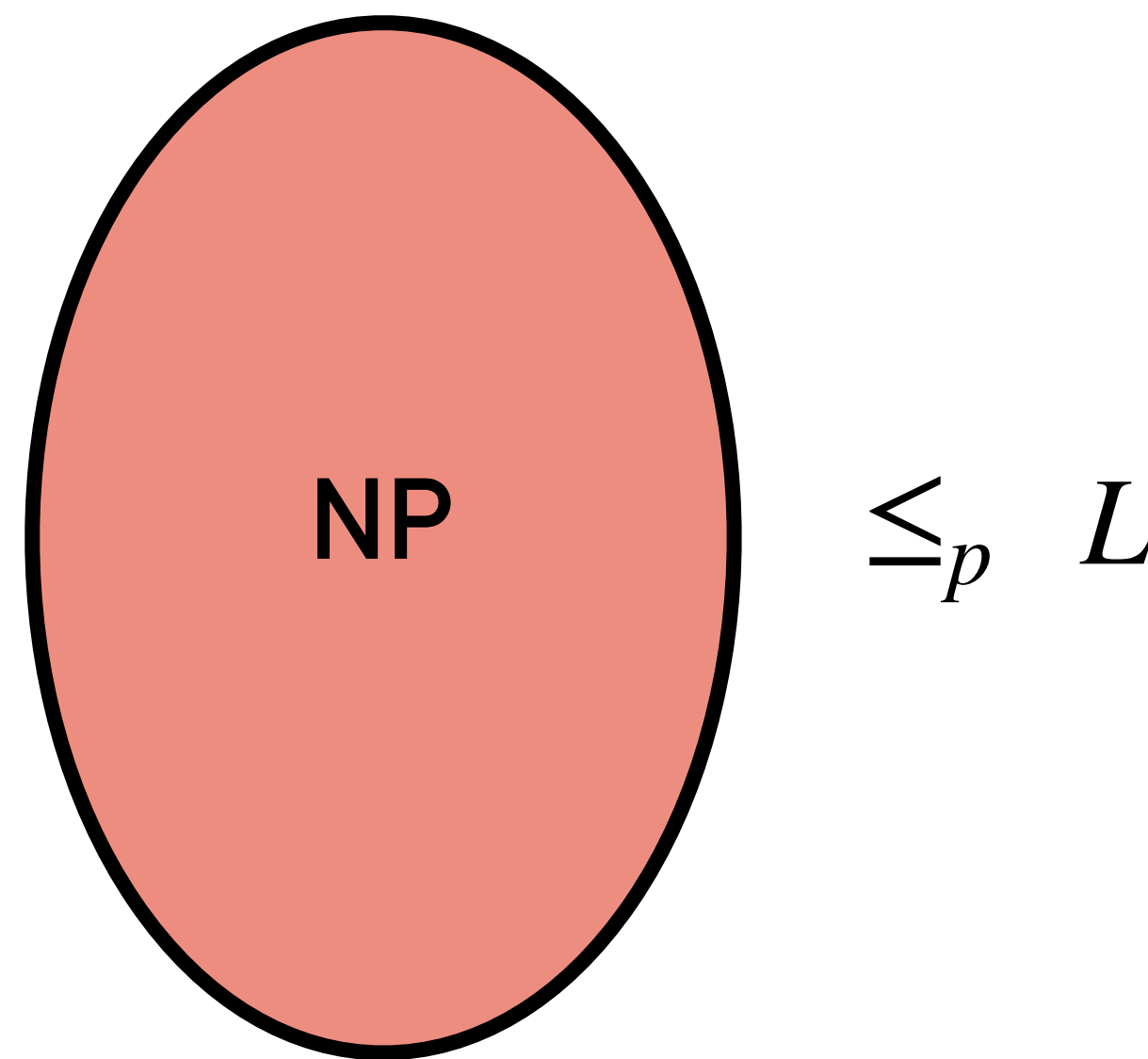
NP-Completeness and NP-Hardness

Defn: A decision problem L in **NP** is called **NP-complete** if L' can be “reduced” to L , denoted by $L' \leq_p L$ in polynomial-time, for every $L' \in \text{NP}$.



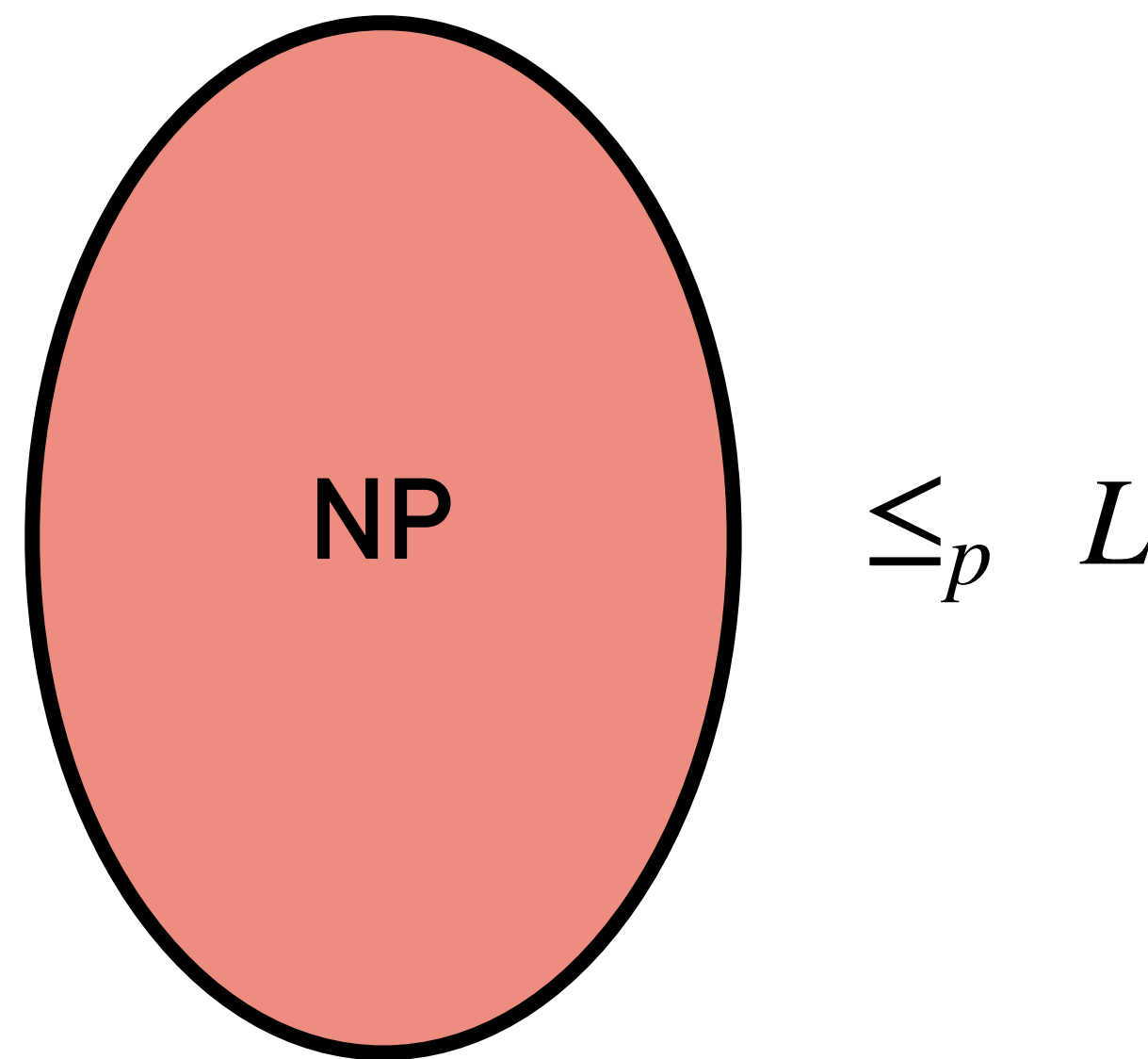
NP-Completeness and NP-Hardness

Defn: A decision problem L in **NP** is called **NP-complete** if L' can be “reduced” to L , denoted by $L' \leq_p L$ in polynomial-time, for every $L' \in \text{NP}$.



NP-Completeness and NP-Hardness

Defn: A decision problem L in **NP** is called **NP-complete** if L' can be “reduced” to L , denoted by $L' \leq_p L$ in polynomial-time, for every $L' \in \text{NP}$.



Observation: If L is polytime solvable, then so is every problem in **NP**.

Cook-Levin Theorem

Cook-Levin Theorem

Cook-Levin Theorem [Coo71, Lev73]:

Cook-Levin Theorem

Cook-Levin Theorem [Coo71, Lev73]:

1) *SAT* is NP-complete.

Cook-Levin Theorem

Cook-Levin Theorem [Coo71, Lev73]:

- 1) *SAT* is NP-complete.
- 2) *3SAT* is NP-complete.

Cook-Levin Theorem

Cook-Levin Theorem [Coo71, Lev73]:

- 1) *SAT* is NP-complete.
- 2) *3SAT* is NP-complete.



$SAT = \{\phi \mid \phi \text{ is a satisfiable CNF formula}\}$

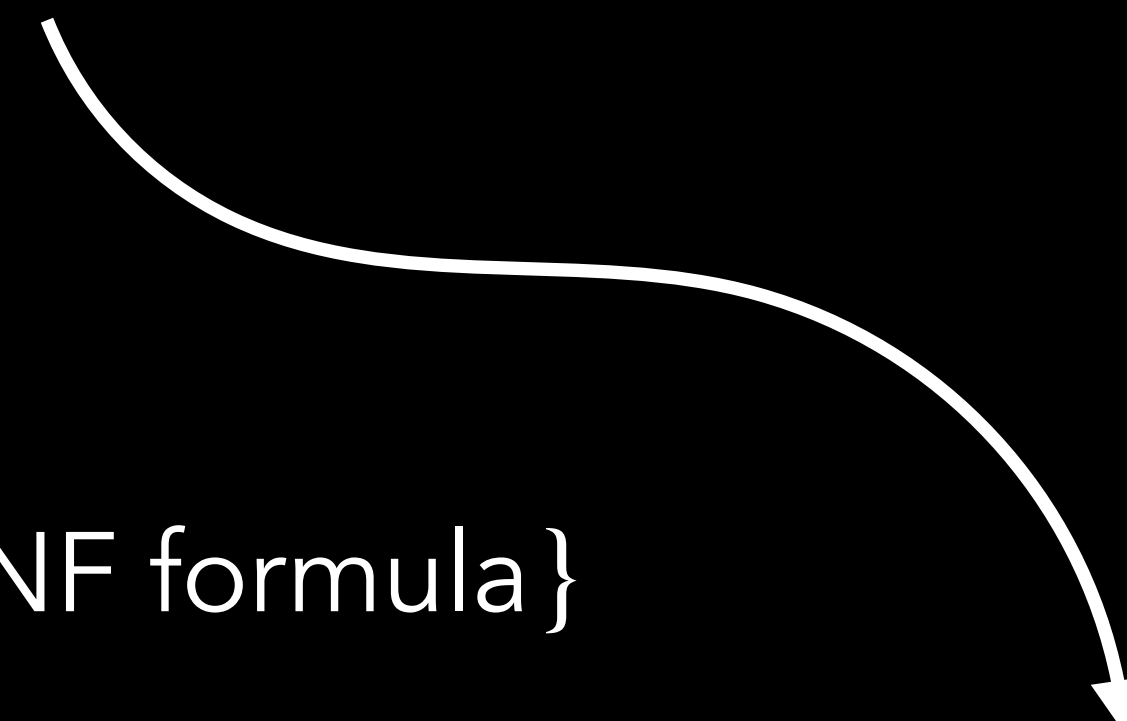
Cook-Levin Theorem

Cook-Levin Theorem [Coo71, Lev73]:

- 1) *SAT* is NP-complete.
- 2) *3SAT* is NP-complete.



SAT = $\{\phi \mid \phi \text{ is a satisfiable CNF formula}\}$



3SAT = $\{\phi \mid \phi \text{ is a satisfiable 3CNF formula}\}$

Boolean Formulas

Boolean Formulas

A **boolean formula** consists of:

Boolean Formulas

A **boolean formula** consists of:

- Variables: $u_1, u_2, \dots, u_k$, where $u_i \in \{0,1\}$.

Boolean Formulas

A **boolean formula** consists of:

- Variables: $u_1, u_2, \dots, u_k$, where $u_i \in \{0,1\}$.
- Operators: AND ($\wedge$), OR ($\vee$), NOT ($\neg$)

Boolean Formulas

A **boolean formula** consists of:

- Variables: $u_1, u_2, \dots, u_k$, where $u_i \in \{0,1\}$.
- Operators: AND ($\wedge$), OR ($\vee$), NOT ($\neg$)

Examples: $\phi_1 = u_1 \wedge \neg u_1$

Boolean Formulas

A **boolean formula** consists of:

- Variables: $u_1, u_2, \dots, u_k$, where $u_i \in \{0,1\}$.
- Operators: AND ($\wedge$), OR ($\vee$), NOT ($\neg$)

Examples: $\phi_1 = u_1 \wedge \neg u_1$, $\phi_2 = (u_1 \vee u_2) \wedge (u_3 \wedge \neg u_4)$

Boolean Formulas

A **boolean formula** consists of:

- Variables: $u_1, u_2, \dots, u_k$, where $u_i \in \{0,1\}$.
- Operators: AND ($\wedge$), OR ($\vee$), NOT ($\neg$)

Examples: $\phi_1 = u_1 \wedge \neg u_1$, $\phi_2 = (u_1 \vee u_2) \wedge (u_3 \wedge \neg u_4)$

Let ϕ be a boolean formula over $u_1, u_2, \dots, u_n$ and $z \in \{0,1\}^n$:

Boolean Formulas

A **boolean formula** consists of:

- Variables: $u_1, u_2, \dots, u_k$, where $u_i \in \{0,1\}$.
- Operators: AND ($\wedge$), OR ($\vee$), NOT ($\neg$)

Examples: $\phi_1 = u_1 \wedge \neg u_1$, $\phi_2 = (u_1 \vee u_2) \wedge (u_3 \wedge \neg u_4)$

Let ϕ be a boolean formula over $u_1, u_2, \dots, u_n$ and $z \in \{0,1\}^n$:

- $\phi(z)$ denotes the value of ϕ when $u_i = z_i$.

Boolean Formulas

A **boolean formula** consists of:

- Variables: $u_1, u_2, \dots, u_k$, where $u_i \in \{0,1\}$.
- Operators: AND ($\wedge$), OR ($\vee$), NOT ($\neg$)

Examples: $\phi_1 = u_1 \wedge \neg u_1$, $\phi_2 = (u_1 \vee u_2) \wedge (u_3 \wedge \neg u_4)$

Let ϕ be a boolean formula over $u_1, u_2, \dots, u_n$ and $z \in \{0,1\}^n$:

- $\phi(z)$ denotes the value of ϕ when $u_i = z_i$.
- ϕ is **satisfiable** if $\exists z$ such that $\phi(z) = 1$. Otherwise, ϕ is **unsatisfiable**.

Boolean Formulas

A **boolean formula** consists of:

- Variables: $u_1, u_2, \dots, u_k$, where $u_i \in \{0,1\}$.
- Operators: AND ($\wedge$), OR ($\vee$), NOT ($\neg$)

Examples: $\phi_1 = u_1 \wedge \neg u_1$, $\phi_2 = (u_1 \vee u_2) \wedge (u_3 \wedge \neg u_4)$

Let ϕ be a boolean formula over $u_1, u_2, \dots, u_n$ and $z \in \{0,1\}^n$:

- $\phi(z)$ denotes the value of ϕ when $u_i = z_i$.
- ϕ is **satisfiable** if $\exists z$ such that $\phi(z) = 1$. Otherwise, ϕ is **unsatisfiable**.

Examples: ϕ_1 is unsatisfiable,

Boolean Formulas

A **boolean formula** consists of:

- Variables: $u_1, u_2, \dots, u_k$, where $u_i \in \{0,1\}$.
- Operators: AND ($\wedge$), OR ($\vee$), NOT ($\neg$)

Examples: $\phi_1 = u_1 \wedge \neg u_1$, $\phi_2 = (u_1 \vee u_2) \wedge (u_3 \wedge \neg u_4)$

Let ϕ be a boolean formula over $u_1, u_2, \dots, u_n$ and $z \in \{0,1\}^n$:

- $\phi(z)$ denotes the value of ϕ when $u_i = z_i$.
- ϕ is **satisfiable** if $\exists z$ such that $\phi(z) = 1$. Otherwise, ϕ is **unsatisfiable**.

Examples: ϕ_1 is unsatisfiable, ϕ_2 is satisfiable as $\phi_2(z) = 1$, for $z = 1010$.

SAT **and** *3SAT*

SAT and 3SAT

Definition: A **CNF formula** is a boolean formula in the form of AND of ORs of variables

SAT and 3SAT

Definition: A **CNF formula** is a boolean formula in the form of AND of ORs of variables or negation of variables.

SAT and 3SAT

Definition: A **CNF formula** is a boolean formula in the form of AND of ORs of variables or negation of variables.

Example: $(u_1 \vee \neg u_2 \vee u_3) \wedge (u_3 \vee u_2 \vee \neg u_1) \wedge (u_2 \vee \neg u_3 \vee u_4)$

SAT and 3SAT

Definition: A **CNF formula** is a boolean formula in the form of AND of ORs of variables or negation of variables.

Example: $(u_1 \vee \neg u_2 \vee u_3) \wedge (u_3 \vee u_2 \vee \neg u_1) \wedge (u_2 \vee \neg u_3 \vee u_4)$



Literals

SAT and 3SAT

Definition: A **CNF formula** is a boolean formula in the form of AND of ORs of variables or negation of variables.

Example: $(u_1 \vee \neg u_2 \vee u_3) \wedge (u_3 \vee u_2 \vee \neg u_1) \wedge (u_2 \vee \neg u_3 \vee u_4)$

Literals



SAT and 3SAT

Definition: A **CNF formula** is a boolean formula in the form of AND of ORs of variables or negation of variables.

Example: $(u_1 \vee \neg u_2 \vee u_3) \wedge (u_3 \vee u_2 \vee \neg u_1) \wedge (u_2 \vee \neg u_3 \vee u_4)$



Literals

SAT and 3SAT

Definition: A **CNF formula** is a boolean formula in the form of AND of ORs of variables or negation of variables.

Example: $(u_1 \vee \neg u_2 \vee u_3) \wedge (u_3 \vee u_2 \vee \neg u_1) \wedge (u_2 \vee \neg u_3 \vee u_4)$



Literals

Clause

SAT and 3SAT

Definition: A **CNF formula** is a boolean formula in the form of AND of ORs of variables or negation of variables.

Example: $(u_1 \vee \neg u_2 \vee u_3) \wedge (u_3 \vee u_2 \vee \neg u_1) \wedge (u_2 \vee \neg u_3 \vee u_4)$



Literals

Clause

Definition: 1) $SAT = \{\phi \mid \phi \text{ is a satisfiable CNF formula}\}$

SAT and 3SAT

Definition: A **CNF formula** is a boolean formula in the form of AND of ORs of variables or negation of variables.

Example: $(u_1 \vee \neg u_2 \vee u_3) \wedge (u_3 \vee u_2 \vee \neg u_1) \wedge (u_2 \vee \neg u_3 \vee u_4)$



Literals

Clause

Definition: 1) $SAT = \{\phi \mid \phi \text{ is a satisfiable CNF formula}\}$

2) $3SAT = \{\phi \mid \phi \text{ is a satisfiable 3CNF formula}\}$

SAT and 3SAT

Definition: A **CNF formula** is a boolean formula in the form of AND of ORs of variables or negation of variables.

Example: $(u_1 \vee \neg u_2 \vee u_3) \wedge (u_3 \vee u_2 \vee \neg u_1) \wedge (u_2 \vee \neg u_3 \vee u_4)$



Literals

Clause

Definition: 1) $SAT = \{\phi \mid \phi \text{ is a satisfiable CNF formula}\}$

2) $3SAT = \{\phi \mid \phi \text{ is a satisfiable 3CNF formula}\}$



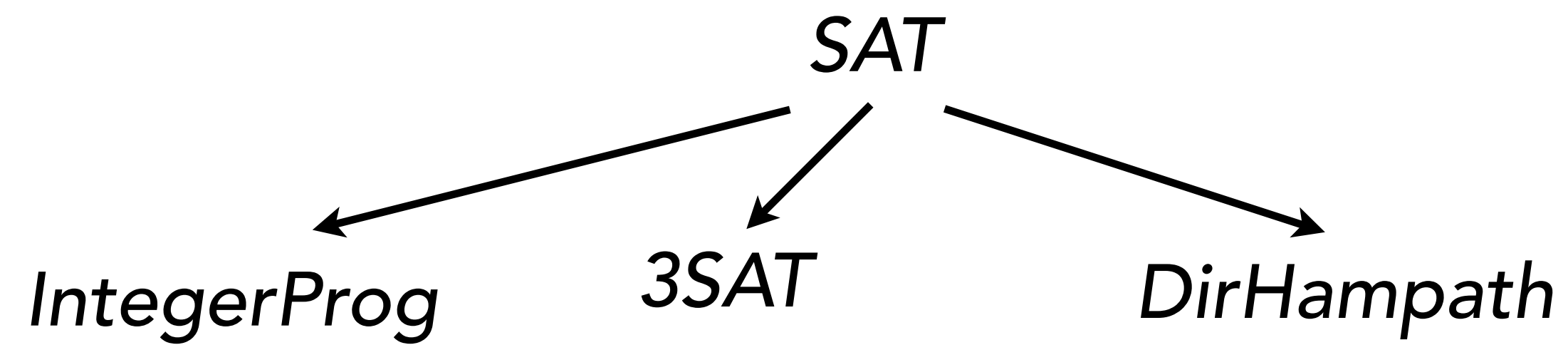
At most 3 literals per clause

NP-complete **Network**

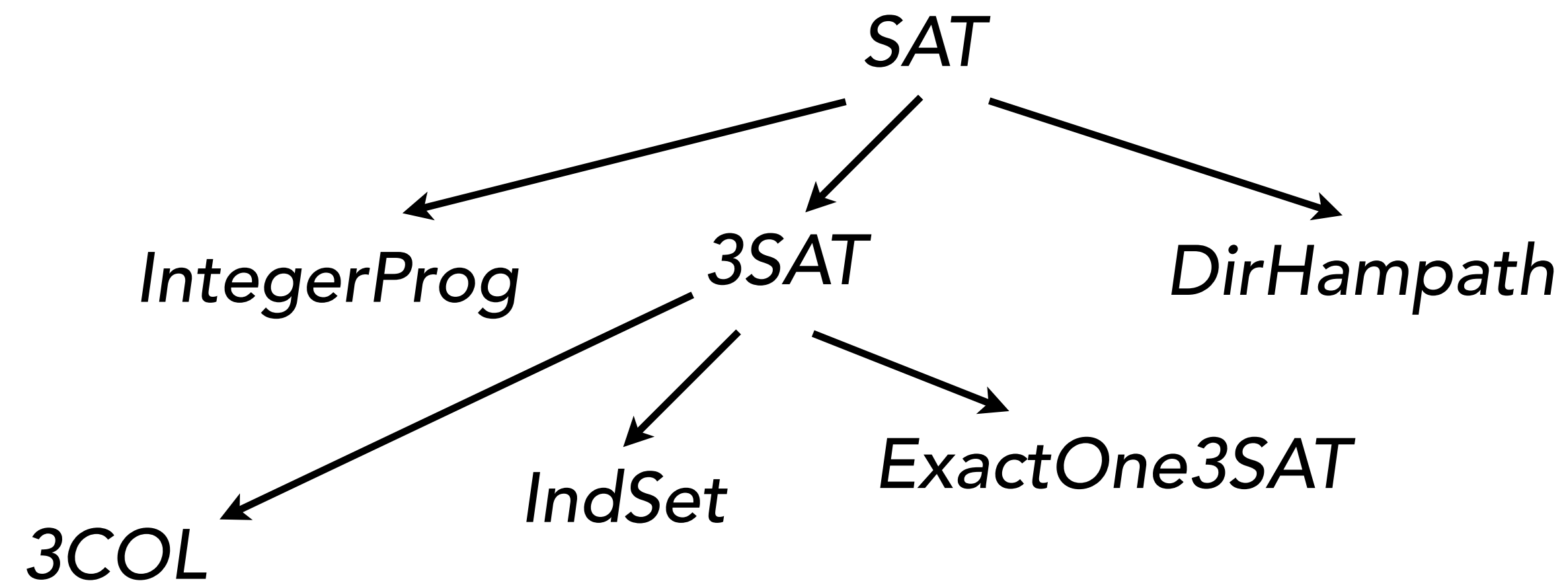
NP-complete **Network**

SAT

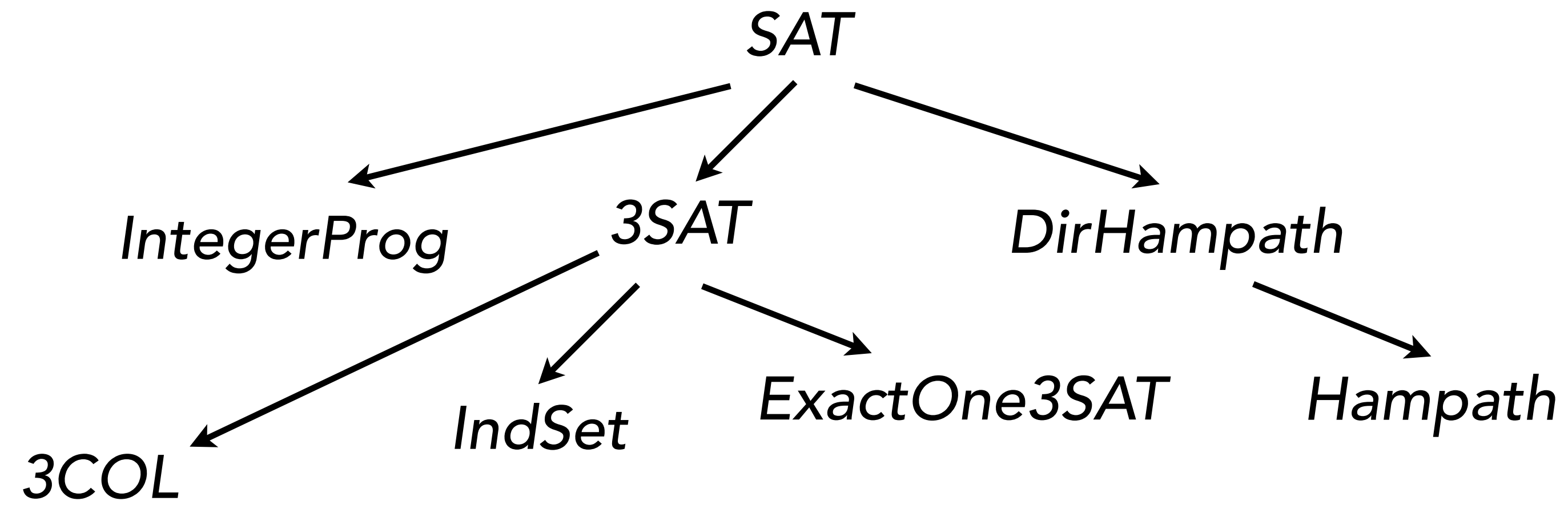
NP-complete **Network**



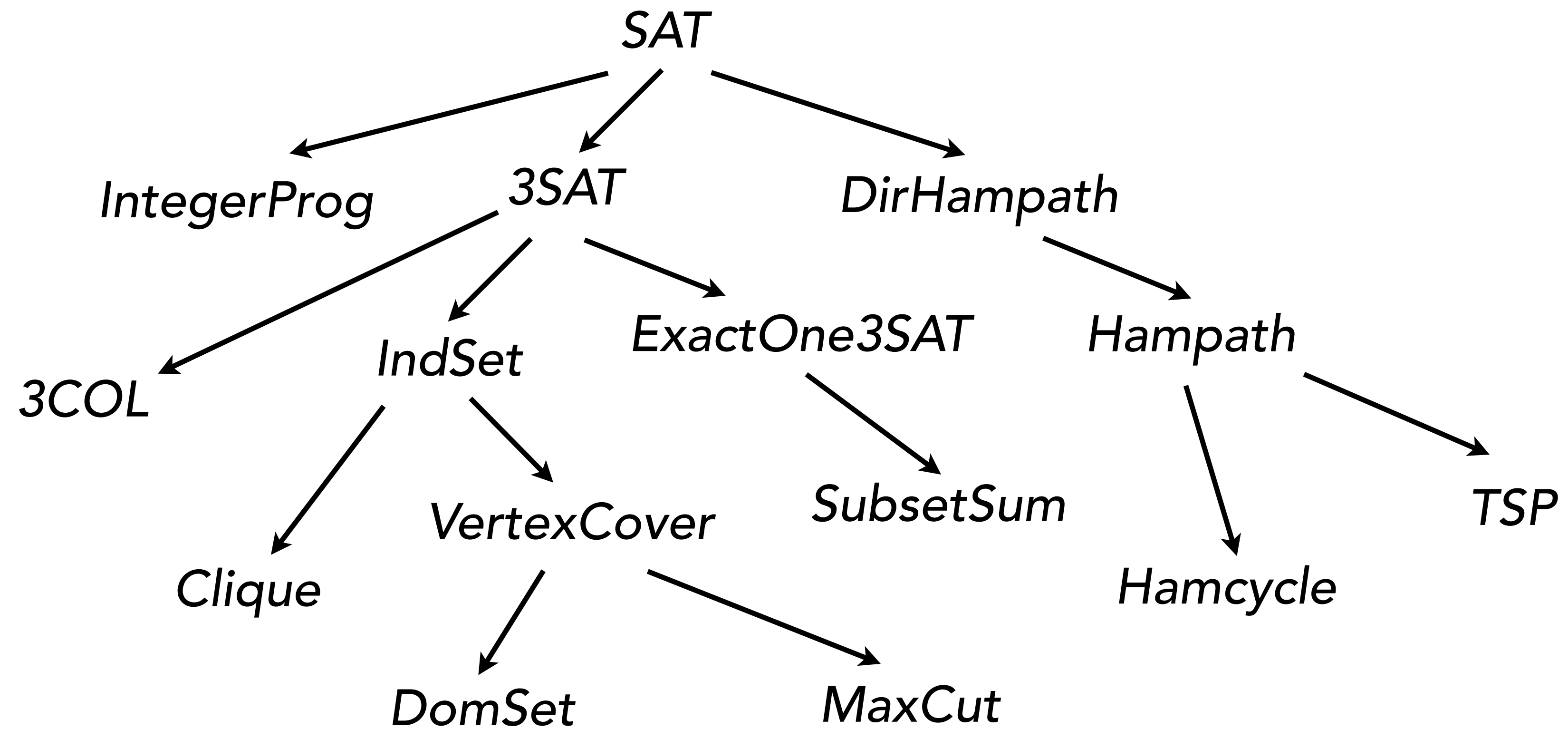
NP-complete **Network**



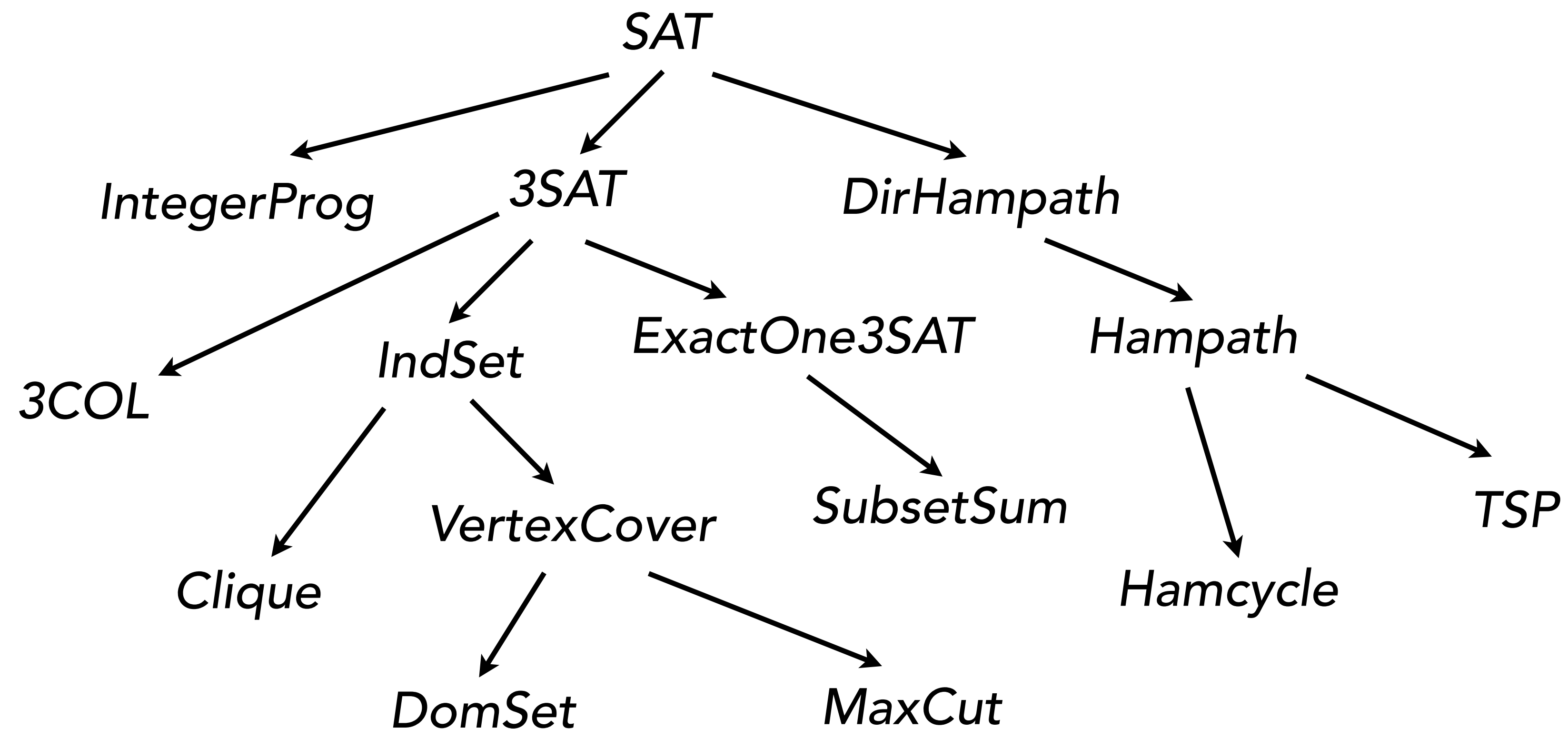
NP-complete **Network**



NP-complete **Network**



NP-complete **Network**



“In this paper we give theorems that suggest, but do not imply, that these problems, as well as many others, will remain intractable perpetually.” – Richard Karp, 1972